Web Workers API

Web Workers makes it possible to run a script operation in a background thread separate from the main execution thread of a web application. The advantage of this is that laborious processing can be performed in a separate thread, allowing the main (usually the UI) thread to run without being blocked/slowed down.

Concepts and usage

A worker is an object created using a constructor (e.g., `Worker()`) that runs a named JavaScript file — this file contains the code that will run in the worker thread.

In addition to the standard JavaScript set of functions (such as `String`, `Array`, `Object`, `JSON`, etc.), you can run almost any code you like inside a worker thread. There are some exceptions: for example, you can't directly manipulate the DOM from inside a worker, or use some default methods and properties of the `Window` object. For information about the code that you *can* run see supported functions, and supported Web APIs.

Data is sent between workers and the main thread via a system of messages — both sides send their messages using the `postMessage()` method, and respond to messages via the `onmessage` event handler (the message is contained within the `message` event's `data` property). The data is copied rather than shared.

Workers may in turn spawn new workers, as long as those workers are hosted within the same origin as the parent page.

In addition, workers can make network requests using the `fetch()` or `XMLHttpRequest` APIs (although note that the `responseXML` attribute of `XMLHttpRequest` will always be `null`).

Worker types

There are a number of different types of workers:

- Dedicated workers are workers that are utilized by a single script. This context is represented by a `DedicatedWorkerGlobalScope` object.
- Shared workers are workers that can be utilized by multiple scripts running in different windows, IFrames, etc., as long as they are in the same domain as the worker. They are a little more complex than dedicated workers — scripts must communicate via an active port.
- Service Workers essentially act as proxy servers that sit between web applications, the browser, and the network (when available). They are intended, among other things, to enable the creation of effective offline experiences, intercept network requests and take appropriate action based on whether the network is available, and update assets residing on the server. They will also allow access to push notifications and background sync APIs.

Worker contexts

While `Window` is not directly available to workers, many of the same methods are defined in a shared mixin (`WindowOrWorkerGlobalScope`), and made available to workers through their own `WorkerGlobalScope`-derived contexts:

- `DedicatedWorkerGlobalScope` for dedicated workers
- `SharedWorkerGlobalScope` for shared workers
- `ServiceWorkerGlobalScope` for service workers

Interfaces

`Worker`

Represents a running worker thread, allowing you to pass messages to the running worker code.

`WorkerLocation`

Defines the absolute location of the script executed by the `Worker`.

`SharedWorker`

Represents a specific kind of worker that can be accessed from several browsing contexts (i.e., windows, tabs, or iframes) or even other workers.

`WorkerGlobalScope`

Represents the generic scope of any worker (doing the same job as `Window` does for normal web content). Different types of worker have scope objects that inherit from this interface and add more specific features.

`DedicatedWorkerGlobalScope`

Represents the scope of a dedicated worker, inheriting from `WorkerGlobalScope` and adding some dedicated features.

`SharedWorkerGlobalScope`

Represents the scope of a shared worker, inheriting from `WorkerGlobalScope` and adding some dedicated features.

`WorkerNavigator`

Represents the identity and state of the user agent (the client).

Examples

We have created a couple of demos to show web worker usage:

- Basic dedicated worker example (run dedicated worker).
- Basic shared worker example (run shared worker).
- OffscreenCanvas worker example (run OffscreenCanvas worker).

You can find out more information on how these demos work in Using Web Workers.

Specifications

Specification
HTML # workers

See also

- Using Web Workers
- `Worker` interface
- `SharedWorker` interface
- Service Worker API

mdn_

Your blueprint for a better internet.